DEEPScreen; Modeling Drug-Target Interactions: Deep Learning Architectures to Classify Drug Activity for Target Proteins

Team Members: Stephen Marriott, Wanming He, William Welsh

Prior GitHub Repository: https://github.com/Stephen-Marriott/CSCI1470-DEEPScreen
Final GitHub Repository: https://github.com/Stephen-Marriott/CSCI1470-DEEPScreen-Final

Introduction

For our final project, our group reimplemented the model outlined in the following paper: *DEEPScreen: high performance drug–target interaction prediction with convolutional neural networks using 2-D structural compound representations.* The goal behind this model is to predict the activity of drug-candidate compounds for target proteins using a convolutional neural network. This is an area of promise in deep learning moving forward, and touches on the intersection of chemistry and health, which all team members involved found interesting. This is structured in the form of a binary classification problem, where the model will determine whether the compound is active or inactive for that specific target. In addition to implementing this model in TensorFlow, our group built two additional model structures to attempt to improve performance at this task. Our team is primarily looking at accuracy, but other metrics like precision are also important, as false positives could result in wasted investment and resources.

Methodology

The DEEPScreen model is trained on data collected from the ChEMBL database. The training data consists of 2d representations of the molecular drug candidates as .png files, along with a classification of active or inactive for each particular protein target. The 2d representation is advantageous over previously used SMILES string representations as they contain considerably more structural information about the molecules, which is an essential component of protein binding.

The DEEPScreen GitHub repository provided a link to a DropBox containing folders for each target protein, as well as images and labels for the active and inactive component compounds. The team has downloaded the data, although not all of it is uploaded to GitHub due to the size and volume. DEEPScreen provides an architecture for a model to learn the protein composition, but each protein has a separate model trained. Our group has re-implemented the DEEPScreen structure in TensorFlow, rather than the original PyTorch, and created two other architectures in the hopes of improving model performance. The first is an alternative CNN, with a smaller number of deeper convolutional layers than the original. The second aims to incorporate spatial attention into the classification model, to better model the spatial structure of the underlying proteins.

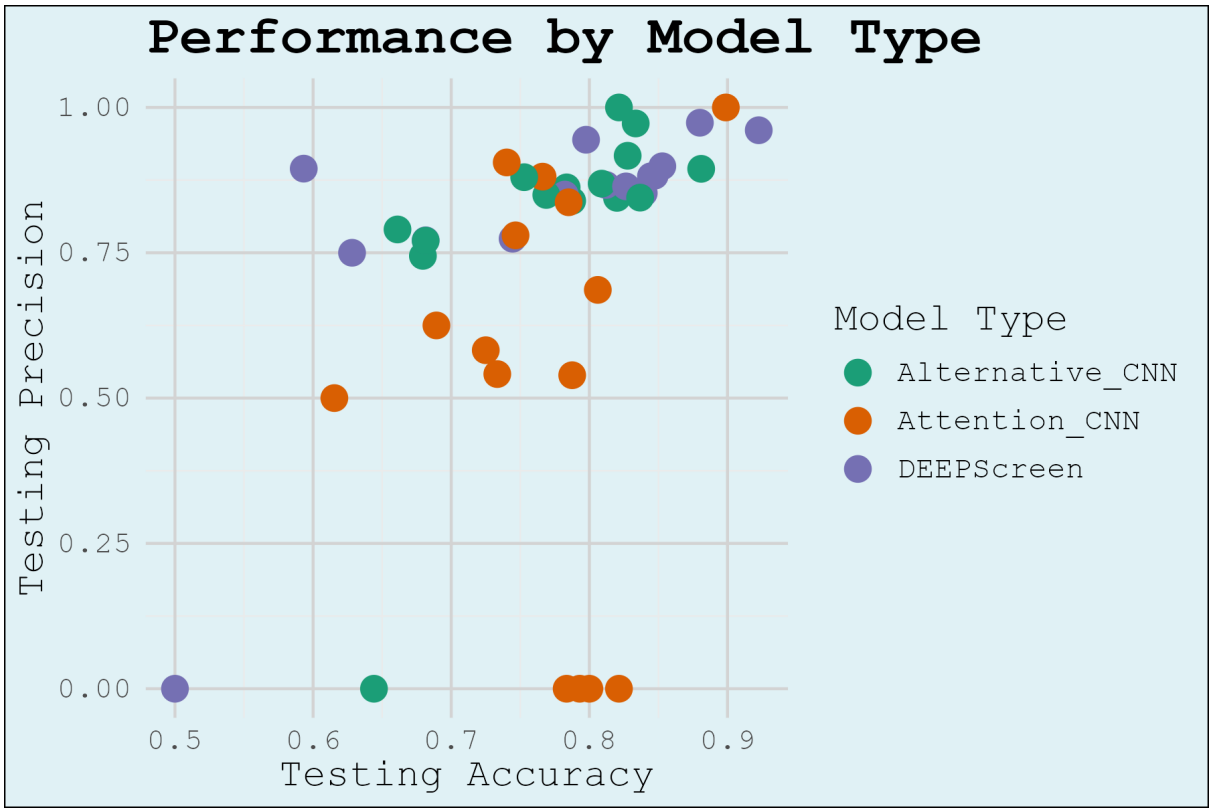
Challenges

Our group has hit a few stumbling blocks along the way. First, a lot of the data processing in the original paper made use of PyTorch, in addition to the original model. This forced us to create new functions to load and split the images based on the provided labels. There were also some issues with the data consistency which were ignored by the original paper: the 2d molecular representations that were generated from the SMILES strings are automatically scaled to fit the window of the .png file, resulting in lower molecular weight molecules appearing larger relative to larger molecular weight molecules. This likely introduces some inaccuracy in the trained models, as atom-to-atom distances are crucial in understanding protein-substrate binding, and these being represented inconsistently throughout the training

data likely causes the model to learn these spatial features incorrectly. Finding a plausible way to implement spatial attention was a challenge, and this version borrows from what was originally a model for 3-Dimensional spatial attention. As we've built three separate model architectures, that has required a lot of time and effort to tune parameters and change the number and shape of layers. In particular, the two models we designed as alternatives to DeepScreen appeared to rapidly overfit the training data, which necessitated some changes in setup, as well as a lowering of the learning rate for the optimizer. While this has made the alternative CNN we proposed a little better than DEEPScreen in the models we've tested so far, we have yet to find a consistently accurate implementation of a model incorporating spatial attention. The attention model also takes a little bit longer to train, and adding more layers or rearranging existing ones has primarily served to slow the training process without adding any accuracy benefits.

Results

We ended up building models of each architecture for 15 proteins, resulting in 45 different models. A scatterplot of the results showing accuracy and precision is displayed below:



The results are also available in a CSV file in the DEEPScreen Implementations folder in the GitHub repository.

The mean and standard deviation of the accuracy by model type for the 15 proteins is shown in the following table:

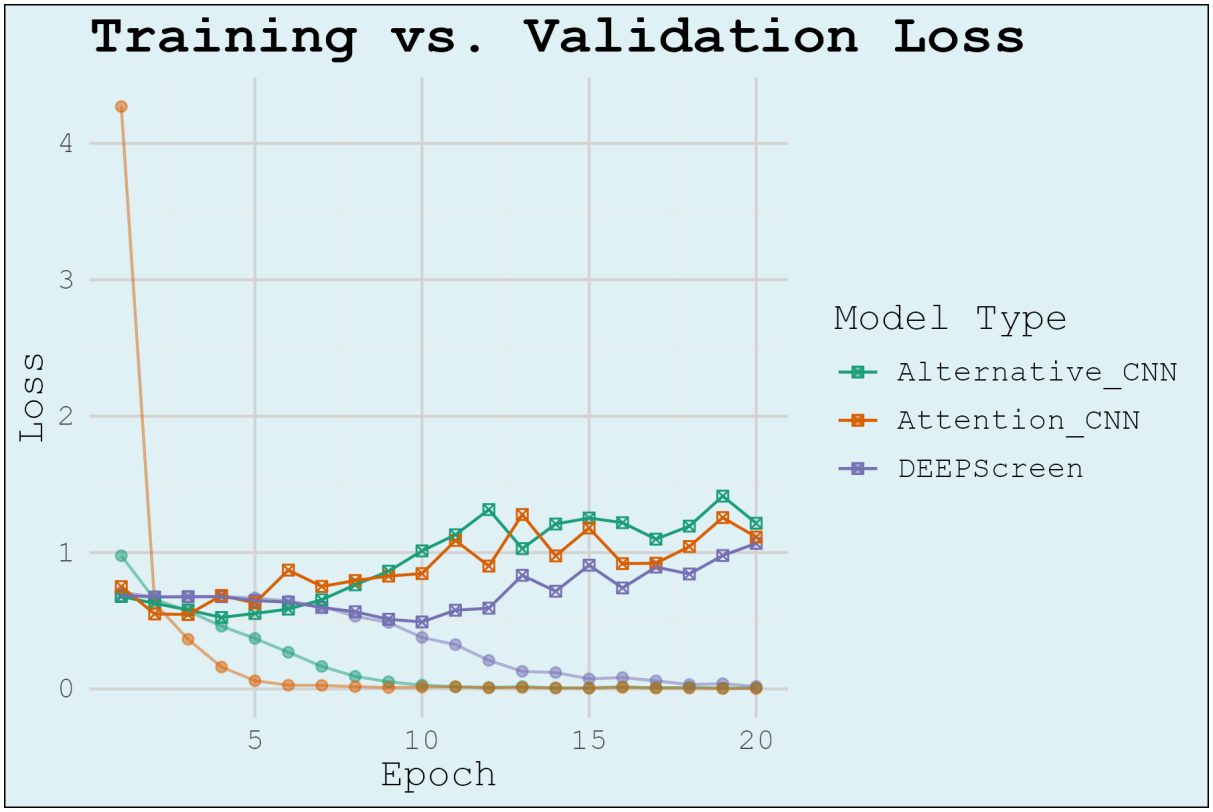
Model Type	Average of Test Accuracy	Std. Dev. Of Test Accuracy
Alternative_CNN	77.25%	7.08%
Attention_CNN	76.61%	6.20%
DEEPScreen	77.02%	11.47%
Grand Total	**76.96%**	**8.57%**

Our Alternative CNN slightly outperformed DEEPScreen in terms of accuracy, and notably had a lower standard deviation in this measure than DEEPScreen, suggesting it was more consistent as a model than DEEPScreen for this task. The Attention based model performed more poorly than the other two by accuracy, although it had a lower standard deviation than the other two models.

Reflection

Overall, the project was successful in reimplementing DEEPScreen in TensorFlow and proposing two alternative architectures. The Alternative CNN consistently outperformed the original DEEPScreen implementation in accuracy and precision, meeting our base goal of replication and our target goal of improvement. However, the Attention CNN underperformed, struggling with overfitting and inconsistent recall-precision trade-offs, falling short of our goal of a robust attention-based model.

We hypothesized that both the Alternative CNN and Attention CNN would outperform DEEPScreen. The Alternative CNN met this expectation, but the Attention CNN's instability was surprising. However, we could see from the accuracy and loss results printed with each training epoch that the attention based model overfit the data rapidly. An example of this is shown in the following scatterplot as well, when all models are trained with the same dropout rate of .25 and learning rate of 0.001. Although we spent some time modifying the architecture and adjusting parameters such as learning and dropout rate, this was ultimately unsuccessful in creating an architecture that consistently outperformed either of the other two models.



Our initial plan was to reimplement DEEPScreen in Tensorflow, rather than PyTorch, and make some tweaks to the structure with hopes of improving accuracy. We ended up designing two new models, in addition to our reimplementation, including one that aimed to utilize attention to improve model performance. Our alternative CNN model, initially overfit

the training data rapidly, but we were able to resolve this by reducing layers and tuning learning rates. While our alternative approach to a more simple CNN was successful at improving on DEEPScreen's performance after some restructuring to avoid overfitting, we found that the attention based approach too rapidly overfit the data to provide any meaningful improvements.

Moving forward, I think it would make sense to put more effort into redesigning the attention based model, as it was clear that it can learn this type of data well. We may also be able to modify the training images to eliminate inconsistencies in molecular size and orientation, and atom coloration. We can also experiment with different attention variants to improve spatial focus without overfitting, and could conduct grid searches for optimal learning rates, batch sizes, and regularization.

Our main takeaways from this project were that CNNs perform well on this data structure, and that more complex models may require more fine-tuning to actually perform well on the testing data. We learned from the Alternative CNN's success that well-tuned, deeper convolutions can outperform complex models like attention when data is limited. We also learned that reproducibility is hard as porting PyTorch to TensorFlow introduced lots of bugs. Finally, it is important to document all preprocessing steps and validate intermediate outputs.